

entrega-portal-fase2

Tech Challenge - Fase 2 - Entrega no portal do aluno (PDF)

Grupo: Oficina Turbo (106)

Aluno: Felipe Ricarte Magalhães

Documento para **entrega no portal** (PDF): repositório, `soat-architecture`, arquitetura e vídeo demonstrativo (≤ 15 min; URL alinhado ao [README](#)). Versão PDF gerada: `entrega-portal-fase2.pdf` (mesma pasta).

1. Repositório GitHub (mesmo da Fase 1)

URL: <https://github.com/ricartefelipe/oficina-springboot-mvp>

Branches no repositório (nomes literais): `main` (estável), `develop` (integração).

Acesso ao avaliador SOAT: o repositório deve estar compartilhado com o usuário `soat-architecture` (convite em *Settings* → *Collaborators*).

Documentação principal: [README.md](#) (objetivos Fase 2, arquitetura, fluxo de deploy, execução local, Kubernetes, Terraform, links Swagger/OpenAPI e vídeo). **Critérios vs. enunciário:** [fase2-concluida.md](#).

2. Desenho da arquitetura e recursos escolhidos

2.1 Visão lógica (aplicação)

- **API** Spring Boot (`/api`): rotas admin (JWT/Keycloak) e públicas (tracking / aprovação).
- **Domínio** em bounded contexts (`cadastros`, `catalogo`, `ordemservico`, `shared`) com portas e adaptadores (hexagonal).
- **Persistência:** PostgreSQL (local: Docker Compose; cloud opcional: RDS via Terraform `enable_rds`).
- **Segurança:** Keycloak (JWT); e-mail via SMTP (MailHog em dev).

Diagramas DDD versionados no repositório:

Figura	Arquivo no repositório
Agregado Ordem de Serviço	docs/ddd/diagrams/ordem-servico-agregado.svg
Event storming (contextos)	docs/ddd/diagrams/event-storming-contextos.svg
Event storming (lousa: C/A/E/P/R)	docs/ddd/diagrams/event-storming-lousa-elementos.svg

Artefatos DDD em texto: [docs/ddd/README.md](#) (Domain Storytelling, dicionário, Event Storming).

Figuras (desenho da arquitetura - DDD):

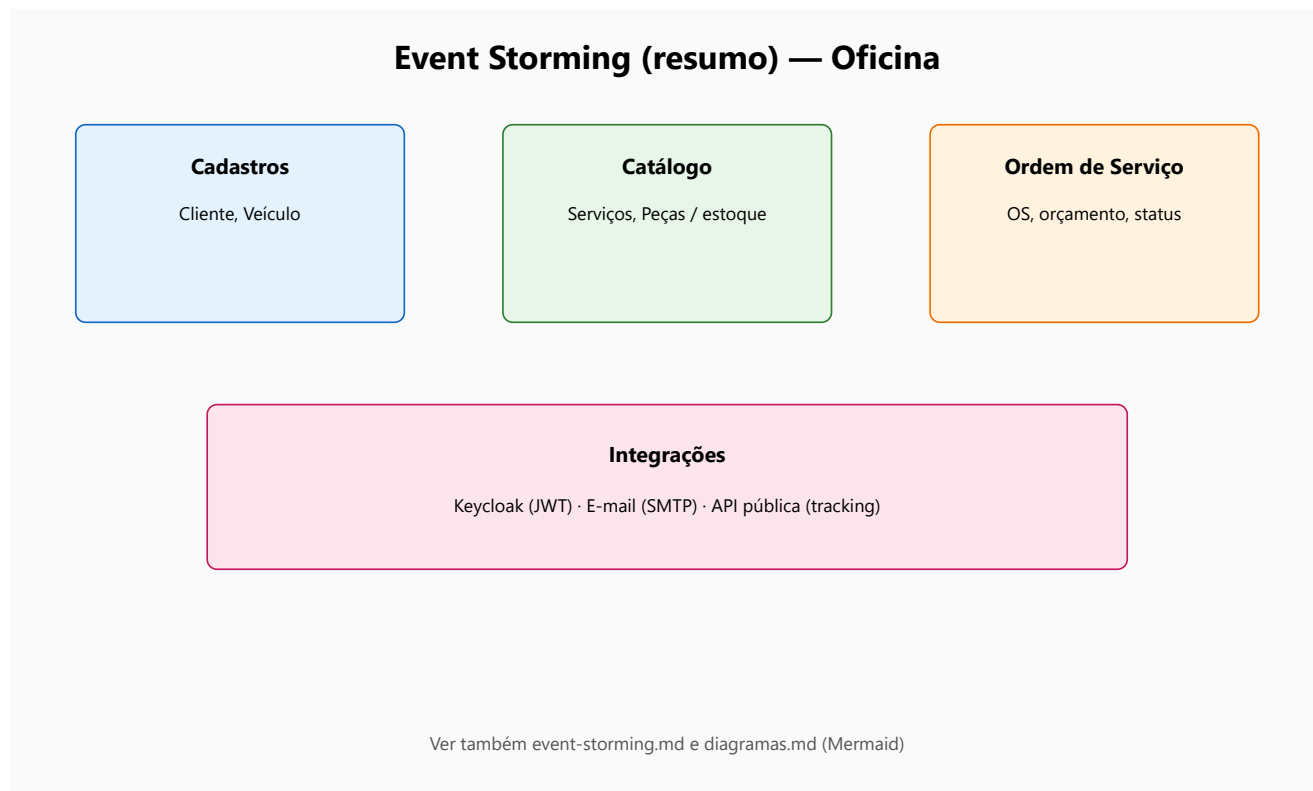
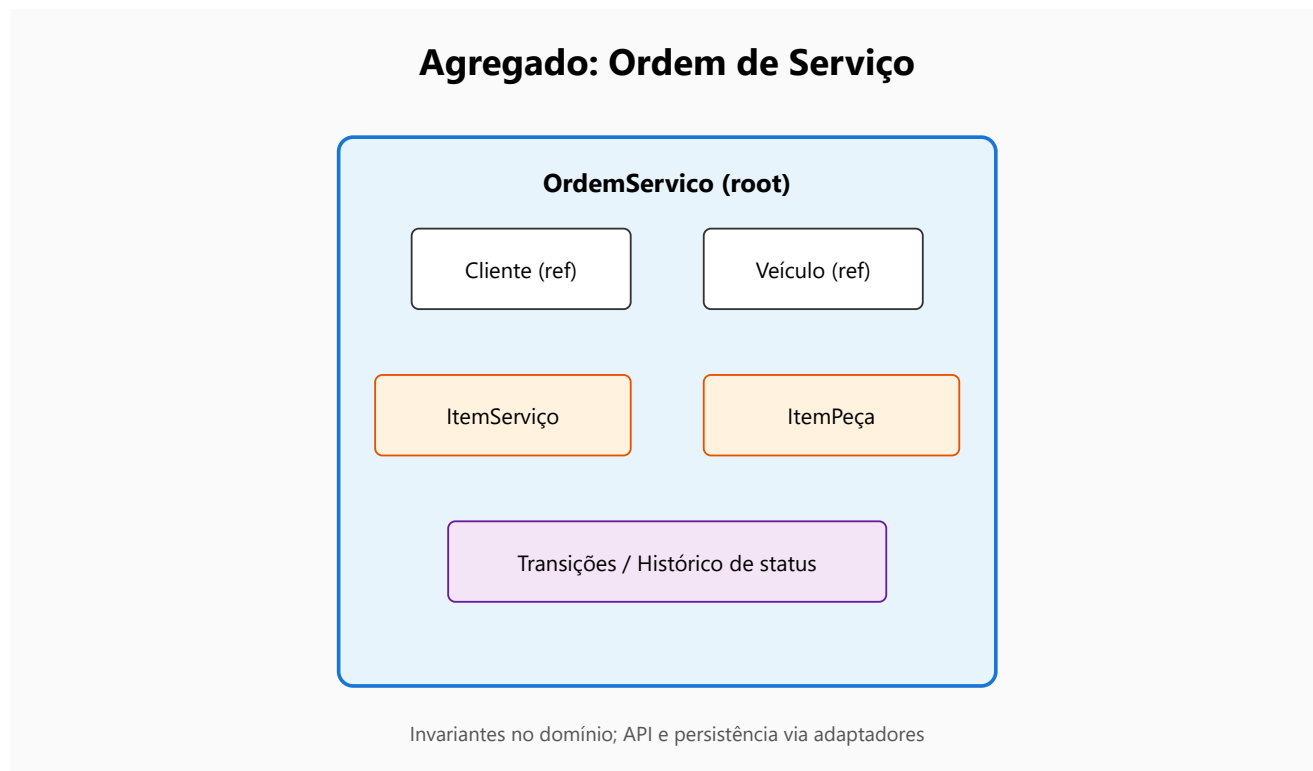


 Diagrama - event storming lousa

2.2 Infraestrutura e automação

Camada	Recurso
Contêineres	<code>Dockerfile</code> , <code>docker-compose.yml</code>
Orquestração	Manifestos em <code>/k8s</code> (Deployment, Service, ConfigMap, Secret, HPA)
IaC	Terraform: <code>/infra</code> (rede AWS; RDS PostgreSQL opcional) e <code>/infra/kind</code> (cluster Kubernetes local com Kind)
CI/CD	GitHub Actions: build Maven, testes, validação Terraform, imagem GHCR; workflows manuais para deploy em Kubernetes e Terraform na AWS

Detalhe do alinhamento ao enunciário (cluster EKS vs. abordagem do repo): [infra/docs/terraform-vs-enunciado.md](#) .

3. Vídeo demonstrativo (≤ 15 min)

Plataforma: YouTube ou Vimeo (público ou não listado).

Link: o mesmo URL publicado na tabela **Links rápidos (APIs e vídeo)** do [README.md](#) do repositório (atualizar o README e regenerar este PDF após publicar).

Conteúdo mínimo (conforme enunciário Fase 2):

1. Deploy da aplicação (Docker Compose e/ou Kubernetes).
2. Execução do CI/CD (ex.: pipeline no GitHub Actions).
3. Consumo das APIs (ex.: Swagger/curl).
4. Escalabilidade automática (ex.: HPA no cluster ou simulação de carga / múltiplas ordens de serviço).

Roteiro sugerido: [docs/video-script.md](#) (Fase 1 + seção Fase 2 no mesmo arquivo).

4. Collection de APIs (Swagger / OpenAPI)

Com a app em `http://localhost:8080` :

- **Swagger UI:** `http://localhost:8080/api/swagger-ui/index.html`
- **OpenAPI JSON:** `http://localhost:8080/api/openapi`

(Postman: Import → Link com o URL do OpenAPI ou arquivo exportado.)

5. Checklist antes de submeter o PDF no portal

Convite `soat-architecture` aceite no repositório

PDF contém **link do GitHub**, **diagramas/arquitetura** e **URL do vídeo** (consistente com o README)

Vídeo com duração $\leq$ **15 minutos** e tópicos do enunciário

[README.md](#) no GitHub com o link do vídeo na tabela **Links rápidos**

Documento complementar: [submission.md](#) (e [submission.pdf](#)).

Regenerar este PDF após editar o Markdown: na raiz do repositório, `.\scripts\delivery\md-to-pdf-edge.ps1 -InputMd "docs\delivery\entrega-portal-fase2.md"` (requer Pandoc e Microsoft Edge).